

Table of Contents

1. Executive Summary	1
2. Phase Scope and Starting Point	2
3. Authentication Core	2
4. Registration and Role Assignment	3
5. Authorization, Ownership and Page Guards	5
6. Role Dashboards and Onboarding Frontend	6
7. Notifications, Audit and Profile Services	7
8. Verification Evidence	7
9. Operational Findings	8
10. Deliberate Deferrals	9
11. Phase Ledger and Next Gate	10

1. Executive Summary

73 Phase 3 verification checks, all passing	227 Total checks across the three suites	7 Role-specific dashboards live	6 Registration journeys shipped
--	---	--	--

Phase 3 of the Dwellers platform turned the authenticated surface on. Building directly on the Phase 1 security foundation and the Phase 2 domain backend, the phase delivered real credential authentication, six registration journeys with server-side role assignment, live account-status enforcement on every request, seven role-specific dashboards behind server-side page guards, and the in-app notification and audit plumbing that surrounds them. The work was performed against a 54-part requirements brief supplied by the project owner, and every part of that brief is accounted for in this report.

Three results define the phase. First, authentication is real: sign-in accepts an email or a Ghana-format phone number, verifies against the bcrypt password hash, and issues a stateless JWT whose userId, role and status claims are captured from the database at sign-in time - never from the client. Second, authorization is live, not decorative: every authenticated request re-reads the account row before it proceeds, so a suspension or deactivation takes effect on the very next request, and dashboard pages resolve their guards from the live database rather than from the session token. Third, onboarding is honest: each of the seven roles lands on a dashboard that shows only what actually exists, with later-phase modules rendered as explicit placeholders rather than fake functionality.

Verification is the third pillar. The new verify-phase3 suite adds 73 checks on top of the Phase 1 (48) and Phase 2 (106) suites, for a combined battery of 227 checks, all green. The suite culminates in a live HTTP section that drives a real NextAuth flow - CSRF handshake, cookie issuance, session claims, password rotation, mid-session suspension, logout - against a running server. A clean-database rebuild test (delete the database, migrate, seed, verify) was executed during the build, and a browser E2E pass walked 16 steps across desktop, mobile and tablet widths with zero console errors. The phase closed with five commits pushed to GitHub and documentation published in AUTHENTICATION.md. The project now sits at the Phase 4 gate, Find / Nationwide Discovery, awaiting the project owner's approval.

2. Phase Scope and Starting Point

2.1 What the phase covers

Phase 3 spans registration, login, sessions, roles, onboarding, profile ownership and dashboards. Everything in that span is enforced server-side; the user interface only reflects decisions the server has already made. The phase also delivers the in-app notification types and audit events that these flows naturally generate, plus the documentation set that lets a future engineer understand the access model without reading every source file.

Equally important is what the phase deliberately does not cover. Quotation, order and payment APIs remain later-phase work - the Phase 2 models and money policy are ready, but no transactional endpoints were opened. Password reset requires an email or SMS delivery channel and is deferred with an honest placeholder page. The administrative verification workflow is a trust-phase concern: Phase 3 records providers as UNVERIFIED and exposes the verification status surfaces, but approval itself belongs to a later phase.

2.2 The starting seam

The PART 1 survey confirmed that Phase 1 had left the auth seam wide open in the best sense: `authOptions.providers` was an empty array awaiting exactly this phase, while the surrounding machinery - the RBAC matrix with escalation defences, the API guard pipeline, the ownership guard, bcrypt password hashing, the audit trail, and the rate limiter - was built, tested and waiting. No foundation was rebuilt or replaced; Phase 3 filled the seam.

The only schema change the phase required was a single boolean: `Business.offersDelivery`, added by migration 20260912050755 and consumed by the supplier dashboard as a delivery-preference control. That the entire authentication phase needed exactly one new column is a direct measure of how much load the Phase 2 domain model absorbed in advance.

3. Authentication Core

3.1 Session architecture

Sessions are stateless JWTs with a seven-day maximum age; there is no session table to provision or expire. The token carries three claims - `userId`, `role` and `status` - and all three are captured from the database row at sign-in time, never accepted from the client. The signing secret is `AUTH_SECRET`, required by the environment module in staging and production. Because the strategy is stateless, role claims refresh on the next token issuance; the phase compensates for status with the live re-check described below.

3.2 Credentials authentication

The credentials provider accepts a single identifier field that may be an email or a Ghana-format phone number, verifies it against the scrypt password hash, and gates on account status before any session is issued. The NextAuth route is wrapped in a POST rate limit of 10 attempts per minute per IP, and outcomes are returned as server-decided codes that the sign-in page maps to friendly copy. Successful sign-ins update lastLoginAt and write a user.login audit event; failures write user.login_failed with the reason recorded and secrets never; sign-out clears the cookie and writes user.logout.

Outcome	Server decision	Behaviour
CredentialsSignin	Identifier or password rejected	Generic incorrect-credentials message; no enumeration hints
ACCOUNT_SUSPENDED	Status gate refuses session	Account-state message; audit event written
ACCOUNT_DEACTIVATED	Status gate refuses session	Account-state message; audit event written
RATE_LIMITED	429 issued before NextAuth	Slow-down message; protects credential stuffing
Success	JWT issued with DB-captured claims	lastLoginAt updated; user.login audited

Table 3.1 - Login outcomes and their server-decided handling

3.3 Live status re-check

The single most consequential security upgrade in the phase lives inside the handler factory: every authenticated request now re-reads the account row - one indexed primary-key select - before the request proceeds. A suspension or deactivation therefore takes effect on the very next request, not at token expiry and not merely by hiding UI elements. The same philosophy drives the page guards on the dashboard side, described in chapter 5, so that both the API surface and the page surface answer to the live database state.

4. Registration and Role Assignment

4.1 Six journeys, one transaction

Registration is path-based: the client submits one of six journeys and the server maps the path to a role through PATH_ROLE_MAP. The request body carries no role field at all, so the client never participates in the decision. Each successful registration creates its full record graph inside one database transaction, writes a user.registered audit event, queues the welcome and profile-completion notifications, and signs the new account in immediately so the user lands on their dashboard without a second form.

Path	Role assigned	Records created (one transaction)
customer	CUSTOMER	User + PersonalProfile
artisan	ARTISAN	User + PersonalProfile + ProviderProfile + ProviderServiceArea(s)
contractor	CONTRACTOR	Same graph as artisan
company	CONSTRUCTION_COMPANY	User + PersonalProfile + Business + BusinessMember (OWNER) + ProviderProfile (business) + BusinessServiceArea(s)
supplier	SUPPLIER	Same graph as company (businessType SUPPLIER)
equipment	EQUIPMENT_PROVIDER	Business created only when a business name is supplied (businessType EQUIPMENT_RENTAL)

Table 4.1 - Registration journeys and the records each creates

4.2 Security properties

The registration surface was built with enumeration and escalation as the primary threats. The properties below were each verified by dedicated checks in the verification suite, and the escalation defences were re-confirmed by the Phase 1 matrix tests that continue to run in the combined battery.

- **Staff roles unreachable by construction.** ADMIN and SUPER_ADMIN are absent from the zod enum; the service double-checks and audits any blocked attempt.
- **Anti-enumeration.** A duplicate email or phone returns one generic conflict message - the response never reveals which identifier collided.
- **Canonical phones.** Ghana-format validation only; numbers are stored normalised to +233 form.
- **Password policy.** Minimum eight characters with upper, lower and digit; hashed with the script utility; never logged and never returned by any API.
- **Email hygiene.** Addresses are lower-cased and trimmed before uniqueness.
- **No self-verification.** Providers start UNVERIFIED; verification is an administrative trust workflow, not a registration side-effect.

5. Authorization, Ownership and Page Guards

5.1 The request pipeline

Every authenticated API request passes through the same ordered pipeline: rate limit, authentication, the live account-status re-check, RBAC against the Phase 1 permission matrix, zod validation, and only then the service layer. The order matters - identity is resolved before authorization, and authorization before input validation - and the Phase 1 architecture tests continue to enforce that ordering in the combined battery.

5.2 Page guards

Dashboard pages do not trust the session token alone. `requireDashboardPage` chains three checks - session validity, a live database read of status and role, and a redirect decision - before any dashboard renders, sending unauthenticated visitors to sign-in and authenticated visitors without access to the dedicated 403 page. The `/dashboard` entry point performs its role redirect from the live database as well, so a stale token cannot strand a user on the wrong dashboard.

5.3 Ownership and IDOR defence

Self-service endpoints are session-scoped: no user identifier travels in any payload, so there is nothing to tamper with. The Phase 2 LOAD, VERIFY, ACT pattern via `assertOwnership` is unchanged and continues to distinguish 404 from 403 correctly - a foreign resource is a 404, never a confirmation that the resource exists.

Endpoint	Scope guarantee
PATCH <code>/api/users/me</code>	Operates on the caller's own account only
PATCH <code>/api/users/me/password</code>	Verifies the current password server-side before changing it
PUT <code>/api/providers/me/service-areas</code>	Replaces the caller's own areas; validates every town id
PUT <code>/api/businesses/me/service-areas</code>	Targets the business owned by the caller only
POST <code>/api/notifications/read</code>	Foreign notification ids return 404, never 200

Table 5.1 - Session-scoped self-service endpoints

6. Role Dashboards and Onboarding Frontend

6.1 Seven dashboards

Each role lands on its own dashboard route, guarded server-side as described in chapter 5. Dashboards expose the surfaces that genuinely exist in this phase - profile, service areas, services, verification status, settings, notifications - and render later-phase modules as honest placeholders. The admin dashboard is staff-only and opens with platform stats, the user table and the audit log.

Route	Roles	Phase 3 contents
/customer	CUSTOMER	Completion widget; later-phase placeholders
/artisan	ARTISAN	Verification banner; real services and service areas
/contractor	CONTRACTOR	Same surfaces as artisan
/company	CONSTRUCTION_COMPANY	Business profile; owner membership
/supplier	SUPPLIER	Business profile plus delivery preference
/equipment	EQUIPMENT_PROVIDER	Business optional, per requirements
/admin	ADMIN, SUPER_ADMIN	Stats, user table, audit log (staff only)

Table 6.1 - Dashboard routes and what each exposes

6.2 Registration wizard

The register entry page presents the six journeys; each journey opens a parameterised wizard with three stages: account credentials, profile details, and location plus service areas. Trades are read live from the category tree rather than hardcoded, and the location cascade ships with no defaults - a user must actively choose region, district and town, which keeps seed data from leaking into production records. The equipment journey makes the business step optional exactly as the brief requires.

6.3 Interface decisions

The sign-in page maps server outcome codes to friendly copy, so ACCOUNT_SUSPENDED reads as an account-state message rather than a stack trace. Header calls-to-action adapt to session state, the mobile navigation uses a sheet at small widths, and a dedicated 403 page explains denied access instead of showing a raw error. Every placeholder states what is coming rather than simulating it - the same honesty policy the project has held since Phase 1.

7. Notifications, Audit and Profile Services

7.1 In-app notifications

The phase ships four notification types - ACCOUNT_WELCOME, PROFILE_COMPLETION, VERIFICATION_STATUS and ACCOUNT_SECURITY - delivered in-app only. The schema already carries channel and sentAt columns, so email, SMS or WhatsApp dispatch can land in a later phase without a migration. List and mark-read endpoints are ownership-scoped; the verification suite proves that marking another user's notifications returns 404 and that mark-all-read touches only the caller's rows.

7.2 Audit trail

Phase 3 adds a complete personal-account audit vocabulary: user.registered, user.login, user.login_failed, user.logout, user.password_changed and user.profile_updated, alongside the Phase 2 actions for provider and business changes. The logger's redaction runs over audit metadata too, so secrets and passwords never reach the trail. The append-only audit_logs table remains the single source for the admin audit page.

7.3 Profile services

The profile bundle endpoint returns the account, its role-specific profile and a completion computation that drives the dashboard widget. Avatar upload reuses the Phase 1 storage policies with SVG explicitly blocked by the BLOCKED_MIME defence, and the password-change endpoint verifies the current password server-side before accepting a replacement. Each of these surfaces has a dedicated ownership check in the verification suite.

8. Verification Evidence

16 / 16 Browser E2E steps passed	0 Console errors across the E2E pass	48 + 106 + 73 Combined battery, all green
--	--	---

The phase's own suite, verify-phase3, runs 73 checks against an isolated database rebuilt from migrations on every run, then finishes against a live server. The battery was re-executed in full during the preparation of this report - lint zero problems, tsc zero errors, and all three suites green on a fresh run - so every number in this chapter reflects the code exactly as it sits in the repository today.

Focus area	Proven behaviour
Registration paths (x6)	Each journey creates its exact record graph and signs the user in
Escalation and input defence	Staff roles rejected; weak passwords and bad phones rejected
Login outcomes	Wrong password, suspended and deactivated accounts handled; rate limit trips
Password change	Current-password verification; re-login with the new password
Profile and ownership	Self-profile ownership; service-area replacement validates towns
Notifications IDOR	Foreign ids 404; mark-all-read scoped to the caller
Live HTTP (NextAuth)	CSRF login, session claims, password rotation, mid-session suspension to 403, logout, cleanup

Table 8.1 - What the Phase 3 suite proves

8.1 Rebuild and browser proofs

Two proofs sit above the suite itself. The clean-database rebuild test deleted the development database, ran migrate deploy from scratch, seeded, brought the server up, and passed all 73 checks against that fresh database - demonstrating that migrations and seed alone reconstruct a working Phase 3. The browser E2E pass then walked 16 steps: customer and artisan registration, sign-in, both dashboards, logout, unauthorized attempts against /admin and /customer landing on 403, a profile edit persisting across reload, and layout checks at mobile 390px and tablet 768px with no overflow - all with zero console errors.

9. Operational Findings

9.1 The AUTH_SECRET provisioning gap

Re-running the live HTTP section in a freshly rebuilt environment exposed a provisioning gap worth recording. The .env file is untracked by design, and the rebuilt environment restored it without AUTH_SECRET. Because the environment module treats the secret as dev-optional, the server booted healthy and every service-level check passed - but NextAuth could not sign session cookies, so all live logins returned 401 with no cookie. Generating a strong secret and restarting the server restored the full battery to green on the next run.

Three lessons follow. First, AUTH_SECRET belongs on the provisioning checklist for every environment, including development. Second, the failure mode surfaces at session issuance rather than at boot, which is exactly where an observability hook belongs in production. Third, the incident vindicates the suite's structure: a purely service-level verification would have passed while real logins were broken, so the live HTTP section is not a luxury but the only check that catches this class of fault.

9.2 Repository hygiene

The same environment rebuild left 149 tracked files with mode-only diffs (100644 to 100755) and no content change. All were restored to their recorded modes and the working tree is clean. The only local commit ahead of origin is the worklog append from the previous session; all project code, at HEAD 69161cf, is pushed.

9.3 Demo account hygiene

The seed creates demo accounts at @demo.dwellers.test sharing one password, every row flagged isSeedData and none ever marked VERIFIED. Seeding remains a manual step and is not part of any deploy path, so these accounts cannot leak into a production database through automation.

10. Deliberate Deferrals

Four limitations are deliberate, documented, and carry a prepared landing spot rather than an apology. Each was scoped with the project owner's requirements rather than discovered by accident, and none blocks the next phase's work. The mid-session role-change case sits adjacent: dashboards re-check the live record on every navigation and the API re-checks status live, while role claims themselves refresh at the next token issuance - a known, bounded window that the documentation states plainly.

Deferral	Why it is deferred	Where it lands
Session revocation after password change	Stateless JWTs stay valid until expiry; immediate revocation needs a token-version column or session table	Explicit PART 42 deferral; schema hook prepared
Password reset	Requires an email or SMS delivery channel	Honest placeholder page today; full flow with the delivery phase
BusinessMember granularity	Phase 3 covers the OWNER membership only	Team-permissions phase
Community-level location FK	Service areas resolve to towns; community areas remain reference data	Discovery refinement in Phase 4+

Table 10.1 - Deferral register

11. Phase Ledger and Next Gate

11.1 Delivery record

The phase shipped as five conventional commits and was pushed to GitHub in full, with AUTHENTICATION.md - a ten-section guide to the access model - joining the documentation set alongside updated README, ARCHITECTURE and DATABASE references. The commit register below is the auditable trail from the Phase 2 boundary to the documentation close.

Commit	Subject
6b354a0	feat: add authentication and registration core
58d398c	chore: ignore tool-results artifacts
270f7ee	feat: add role-based onboarding and dashboards
8b266fb	fix: browser E2E findings and UI polish
69161cf	docs: document authentication architecture

Table 11.1 - Phase 3 commit register (pushed f28f850..69161cf)

11.2 Project phase ledger

Phase	Scope	Status
1	Technical foundation, architecture and standards	COMPLETE - gate PASS WITH CONDITIONS, all conditions resolved
2	Database schema and backend foundation	COMPLETE - 41 models, 9 API groups, 106 checks
3	Authentication, registration and role-based dashboards	COMPLETE - 73 checks, live NextAuth proven, E2E 16/16
4	Find / Nationwide Discovery	NOT STARTED - awaiting project owner approval

Table 11.2 - Platform phases as of this report

The platform now stands at the Phase 4 gate. The discovery phase inherits a fully authenticated user base, service areas already attached to providers and businesses, the category tree and nationwide location hierarchy seeded, and a verification culture that expects every claim in a report to be re-runnable on demand. Per the project's own gate rules, Phase 4 does not begin without the project owner's explicit approval.

11.3 What Phase 4 inherits

The handover state is concrete. Identity is solved: every future surface can call `getAuthContext` and know exactly who is asking, with what role, and whether the account is still live at the moment of the request. The discovery raw material is already in the database - providers and businesses carry their service areas, the category tree is seeded, and the sixteen-region location hierarchy with its districts, towns and community areas is populated and indexed for the search patterns Phase 4 will need. The operational rails are warm as well: the rate limiter, audit trail, live status re-check and ownership guards all run on every request, so new discovery endpoints inherit the same guarantees without new plumbing. What remains for the owner is a single decision - approve Phase 4 - and the same discipline that carried the first three phases will carry the fourth.

Authentication was the last promise the interface could not keep on its own. From here, every screen can answer who is asking - and the next phase is about what they will find.